

LECTURE 4

Operating System Structure and Design

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Mechanism vs. policy — a foundational OS design distinction, and why it matters
- The monolithic structure — simple and fast, but fragile
- The layered approach, and the strict ordering constraints it imposes
- The microkernel — pushing almost everything out into user-space services
- The modular approach — loadable kernel modules, what most real systems use today
- Hybrid systems, and a side-by-side comparison of all four core structures

Design and Implementation Challenges

- There is no single “correct” way to structure an operating system
- The right structure depends on what the system has to do — general-purpose desktop, embedded controller, real-time industrial system
- The goals and specifications set at the very start shape everything that follows
- Early structural decisions are the hardest to change later, once thousands of other things are built on top of them

Mechanism vs. Policy

One distinction, more than any other, determines whether an OS design ages well.

Mechanism — how

A hardware timer that can interrupt the CPU; a memory-protection scheme that can mark a page read-only

Policy — what, when

Which process the scheduler picks to run next; how long a time slice lasts; who gets write access

A mnemonic worth keeping

Mechanism is the engine; policy is the steering. You can swap the driver's decisions — turn left here, slow down there — endlessly, without ever touching how the engine converts fuel into motion.

Why Bother Separating Mechanism From Policy?

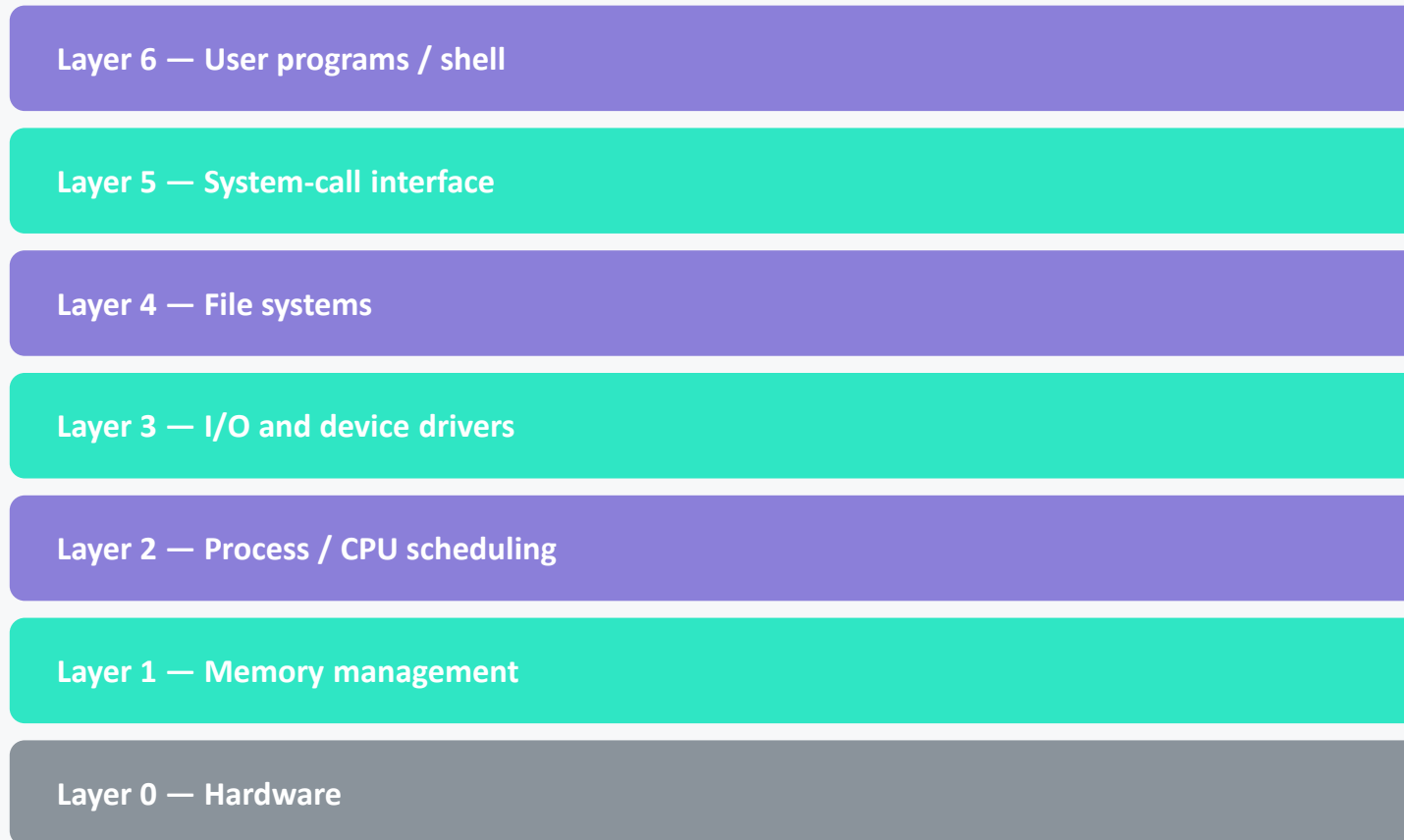
- **Policies change constantly** — across installations, across users, across time
- **A correctly built mechanism should not need to change at all** — it should stay stable underneath whatever policy sits on top
- **Example** — a general, flexible timer mechanism can support round-robin scheduling today and a priority-based policy tomorrow without being touched
- **Mix them together, and** — every policy change risks breaking the underlying machinery it depends on

The Monolithic Structure

- **Everything in one program** — process scheduling, memory management, the file system, device drivers, all in a single address space
- **Early MS-DOS** — written this way out of necessity; limited hardware space forced functionality into the smallest footprint, with no separation of interfaces
- **Original UNIX** — split into a kernel (reachable only through system calls) and system programs on top, but the kernel's own internals stayed one undivided body of code
- **Fast but fragile** — no boundary to cross between subsystems, but a bug anywhere in the kernel can corrupt memory or crash the entire machine

The Layered Approach

Organizes the OS into a stack of layers, 0 at the bottom (hardware) to N at the top (user interface). Each layer may use only the services of layers strictly below it.



Layered Approach: Benefits and Costs

- **Simplicity in construction and debugging** — implement and fully debug layer 0 first in isolation; build layer 1 on top, trusting layer 0 completely; and so on
- **Bugs are easy to localize** — if a bug appears while testing layer k, it must be in layer k itself, since every layer below has already been verified
- **Information hiding** — each layer hides its own data structures and operations from everything above it
- **The cost: overhead and rigidity** — a request filters down through every intermediate layer; discovering layer 3 needs something layer 5 provides means redesigning the stack

The Microkernel

- **The opposite strategy** — strip the kernel to the smallest possible core: minimal process and memory management, plus message-passing
- **Everything else moves out** — file systems, device drivers, and networking run as separate user-space processes, communicating purely through messages
- **Mach** — the textbook example; underlies parts of the modern macOS/iOS kernel
- **Pays off in extensibility, portability, and reliability** — a bug in a user-space service can crash and restart without taking the whole OS down

Monolithic vs. Microkernel Placement of Services

Monolithic

Scheduler, file system, drivers, and networking all run inside one kernel address space

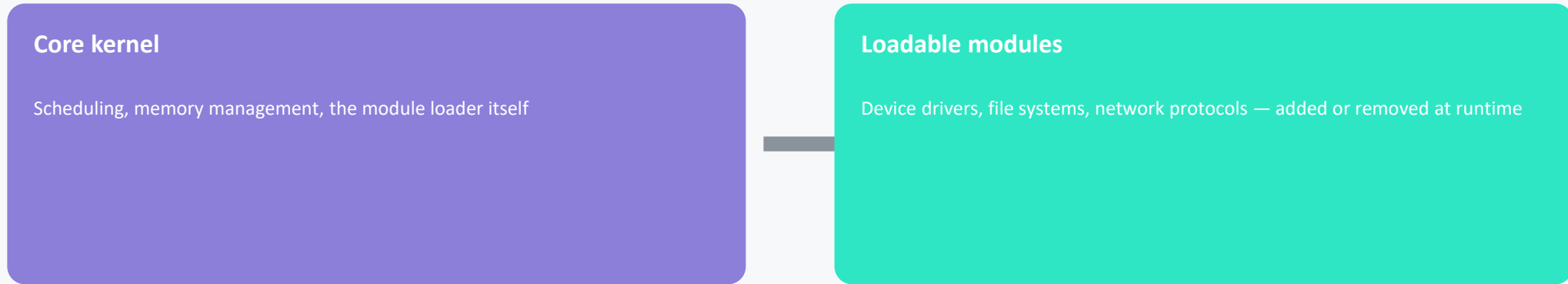
Microkernel

A tiny kernel (scheduling + IPC only), with file system, drivers, and networking each a separate user-space service, reached by message passing

The cost of shrinking the kernel: performance. What used to be a single function call becomes a round trip of message passing, multiplying context switches for a single request.

The Modular Approach

Most OSes shipped today — Linux and Solaris among them — use a modular design built around loadable kernel modules.



Resembles the layered approach's separation of functionality, but without strict one-directional ordering — any module can call any other directly. Resembles the microkernel's independent loadability, but modules share the kernel's own address space, avoiding the message-passing performance tax.

Hybrid Systems: Pragmatic Real-World Mixes

In practice, almost no production OS is a pure example of any single structure — most real systems mix approaches wherever pragmatic.

macOS / iOS (XNU)

Mach microkernel core + monolithic-style BSD components (networking, file system, process model) for performance

Windows (NT)

Partly layered, with performance-critical subsystems pulled into the kernel, plus extensive loadable-driver support

Linux

Monolithic core for speed, extended almost entirely through loadable kernel modules

A pure microkernel is elegant but slow; a pure monolithic kernel is fast but fragile — real engineering teams pick whatever mixture delivers acceptable performance without sacrificing maintainability.

Comparing the Four Structures

	Monolithic	Layered	Microkernel	Modular
Coupling	Tight — one address space, no boundaries	Strict, one-directional — depends only on the layer below	Loose — communicate only via message passing	Loose but direct — modules call each other within one address space
Performance	Fastest — no cross-boundary overhead	Moderate — filters through every layer	Slowest — message passing, extra context switches	Fast — near-monolithic, shared address space
Extensibility	Poor — risks the whole kernel	Moderate — constrained by strict ordering	Excellent — a new service is just a user-space program	Excellent — load/unload without rebuilding
Fault isolation	Poor — a bug can corrupt the whole kernel	Moderate — confined to the layer under test	Excellent — a failing service can often restart	Poor to moderate — a bad module can still crash it
Real-world examples	Early MS-DOS, original UNIX	Early layered research systems (e.g. THE OS)	Mach (parts of macOS/iOS)	Linux, Solaris

Four Clean Categories, Almost No Clean Real-World Examples

Use the table for trade-offs, not classification

This table describes four idealized structures precisely because almost no shipping OS matches any single row exactly — Linux, Windows, and macOS are all, in practice, hybrids that borrow traits from several rows at once. Use the table to reason about trade-offs (what do you gain and lose by leaning monolithic vs. microkernel?), not as a strict classification of any specific real system.

Key Takeaways

- Separating mechanism (how something is done) from policy (what is decided) lets an OS change its policies without rewriting the machinery they rely on
- A monolithic kernel is fast but fragile: no internal boundaries means one bug can corrupt the whole system
- A layered OS is easy to build and debug one layer at a time, at the cost of strict ordering and request overhead
- A microkernel pushes almost everything into user-space services reached by message passing — more robust and extensible, but slower
- A modular kernel (Linux, Solaris) loads independent modules at runtime, combining separation with near-monolithic performance
- Virtually every real OS — macOS/iOS, Windows, Linux included — is a hybrid that mixes these approaches pragmatically